

```

In [ ]: # =====
# 1. IMPORT LIBRARIES
# =====
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.sequence import pad_sequences

# =====
# 2. LOAD DATASET (IMDB)
# =====
vocab_size = 10000

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.imdb.load_data(num_words=v

print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))

# =====
# 3. PREPROCESSING
# =====
max_len = 200

x_train = pad_sequences(x_train, maxlen=max_len, padding='post')
x_test = pad_sequences(x_test, maxlen=max_len, padding='post')

# =====
# 4. BUILD RNN MODEL (FIXED)
# =====
model = models.Sequential([

    # Input Layer (fixes unbuilt issue)
    layers.Input(shape=(max_len,)),

    # Embedding Layer
    layers.Embedding(input_dim=vocab_size, output_dim=128),

    # Simple RNN Layer
    layers.SimpleRNN(64),

    # Dense Layer
    layers.Dense(64, activation='relu'),

    # Dropout
    layers.Dropout(0.5),

    # Output Layer
    layers.Dense(1, activation='sigmoid')
])

# Now summary will show correct parameters
model.summary()

```

```

# =====
# 5. COMPILE MODEL
# =====
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# =====
# 6. TRAIN MODEL
# =====
history = model.fit(
    x_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

# =====
# 7. EVALUATE MODEL
# =====
loss, accuracy = model.evaluate(x_test, y_test)

print("\nTest Accuracy:", accuracy * 100)

# =====
# 8. PREDICT NEW TEXT
# =====
word_index = tf.keras.datasets.imdb.get_word_index()

def encode_text(text):
    words = text.lower().split()
    encoded = [word_index.get(word, 2) for word in words]
    padded = pad_sequences([encoded], maxlen=max_len, padding='post')
    return padded

def predict_sentiment(text):
    processed = encode_text(text)
    prediction = model.predict(processed)[0][0]

    print("\nReview:", text)

    if prediction >= 0.5:
        print("Sentiment: Positive 😊")
    else:
        print("Sentiment: Negative 😞")

# Example predictions

```

```
predict_sentiment("This movie was excellent and amazing")
predict_sentiment("This movie was very bad and boring")
```

Training samples: 25000

Testing samples: 25000

Model: "sequential\_3"

| Layer (type)             | Output Shape     | Param #   |
|--------------------------|------------------|-----------|
| embedding_3 (Embedding)  | (None, 200, 128) | 1,280,000 |
| simple_rnn_3 (SimpleRNN) | (None, 64)       | 12,352    |
| dense_6 (Dense)          | (None, 64)       | 4,160     |
| dropout_3 (Dropout)      | (None, 64)       | 0         |
| dense_7 (Dense)          | (None, 1)        | 65        |

Total params: 1,296,577 (4.95 MB)

Trainable params: 1,296,577 (4.95 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20  
625/625 ————— 14s 17ms/step - accuracy: 0.4974 - loss: 0.7047 - val\_accuracy: 0.5096 - val\_loss: 0.6929  
Epoch 2/20  
625/625 ————— 10s 16ms/step - accuracy: 0.5149 - loss: 0.6955 - val\_accuracy: 0.5376 - val\_loss: 0.6908  
Epoch 3/20  
625/625 ————— 10s 16ms/step - accuracy: 0.5343 - loss: 0.6854 - val\_accuracy: 0.5066 - val\_loss: 0.6899  
Epoch 4/20  
625/625 ————— 11s 17ms/step - accuracy: 0.5782 - loss: 0.6567 - val\_accuracy: 0.5574 - val\_loss: 0.6659  
Epoch 5/20  
625/625 ————— 10s 17ms/step - accuracy: 0.6097 - loss: 0.6088 - val\_accuracy: 0.5616 - val\_loss: 0.6629  
Epoch 6/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6362 - loss: 0.5770 - val\_accuracy: 0.5800 - val\_loss: 0.6741  
Epoch 7/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6617 - loss: 0.5585 - val\_accuracy: 0.5944 - val\_loss: 0.6589  
Epoch 8/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6546 - loss: 0.5434 - val\_accuracy: 0.5922 - val\_loss: 0.6914  
Epoch 9/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6879 - loss: 0.5175 - val\_accuracy: 0.6002 - val\_loss: 0.6971  
Epoch 10/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6854 - loss: 0.4861 - val\_accuracy: 0.6020 - val\_loss: 0.7170  
Epoch 11/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6961 - loss: 0.4598 - val\_accuracy: 0.6118 - val\_loss: 0.7889  
Epoch 12/20  
625/625 ————— 10s 17ms/step - accuracy: 0.7113 - loss: 0.4447 - val\_accuracy: 0.6100 - val\_loss: 0.8226  
Epoch 13/20  
625/625 ————— 10s 16ms/step - accuracy: 0.7087 - loss: 0.4416 - val\_accuracy: 0.7412 - val\_loss: 0.8314  
Epoch 14/20  
625/625 ————— 10s 16ms/step - accuracy: 0.7452 - loss: 0.4377 - val\_accuracy: 0.6092 - val\_loss: 0.8810  
Epoch 15/20  
625/625 ————— 10s 16ms/step - accuracy: 0.7060 - loss: 0.4404 - val\_accuracy: 0.6022 - val\_loss: 0.8399  
Epoch 16/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6989 - loss: 0.4567 - val\_accuracy: 0.6064 - val\_loss: 0.7852  
Epoch 17/20  
625/625 ————— 10s 16ms/step - accuracy: 0.7094 - loss: 0.4245 - val\_accuracy: 0.6076 - val\_loss: 0.9076  
Epoch 18/20  
625/625 ————— 10s 16ms/step - accuracy: 0.6997 - loss: 0.4430 - val\_accuracy: 0.6064 - val\_loss: 0.8764  
Epoch 19/20  
625/625 ————— 10s 16ms/step - accuracy: 0.7138 - loss: 0.4115 - val\_accuracy: 0.7138 - val\_loss: 0.4115

ccuracy: 0.6040 - val\_loss: 0.9274

Epoch 20/20

**625/625**  **10s** 16ms/step - accuracy: 0.7120 - loss: 0.4213 - val\_a

ccuracy: 0.6038 - val\_loss: 0.9763

**782/782**  **4s** 5ms/step - accuracy: 0.5962 - loss: 0.9724

Test Accuracy: 59.623998403549194

**1/1**  **0s** 385ms/step

Review: This movie was excellent and amazing

Sentiment: Negative 😞

**1/1**  **0s** 36ms/step

Review: This movie was very bad and boring

Sentiment: Negative 😞